2025 ACM/IEEE 28th International Conference on Model Driven Engineering Languages and Systems (MODELS) | 979-8-3315-4910-7/25/$31.00 ©2025 IEEE | DOI: 10.1109/MODELS67397.2025.00014

MCeT: Behavioral Model Correctness Evaluation using Large Language Models

Khaled Ahmed
Huawei Research Canada
khaled.ahmed4@h-partners.com

Jialing Song
Huawei Research Canada
jackie.song@h-partners.com

Boqi Chen*
McGill University
boqi.chen@mail.mcgill.ca

Ou Wei
Huawei Research Canada
ou.wei1@huawei.com

Bingzhou Zheng
Huawei Research Canada
bingzhou.zheng@huawei.com

***Abstract*—Behavioral model diagrams, e.g., sequence diagrams, are an essential form of documentation that are typically designed by system engineers from requirements documentation, either fully manually or assisted by design tools. With the growing use of Large Language Models (LLM) as AI modeling assistants, more automation will be involved in generating diagrams. This necessitates the advancement of automatic model correctness evaluation tools. Such a tool can be used to evaluate both manually and AI automatically generated models; to provide feedback to system engineers, and enable AI assistants to self-evaluate and self-enhance their generated models.**

In this paper, we propose MCeT, the first fully automated tool to evaluate the correctness of a behavioral model, sequence diagrams in particular, against its corresponding requirements text and produce a list of issues that the model has. We utilize LLMs for the correctness evaluation tasks as they have shown outstanding natural language understanding ability. However, we show that directly asking an LLM to compare a diagram to requirements finds less than 35% of issues that experienced engineers can find. We propose to supplement the direct check with a fine-grained, multi-perspective approach; we split the diagram into *atomic*, non-divisible interactions, and split the requirements text into atomic, self-contained items. We compare the diagram with atomic requirements and each diagram-atom with the requirements. We also propose a self-consistency checking approach that combines perspectives to mitigate LLM hallucinated issues. Our combined approach improves upon the precision of the direct approach from 0.58 to 0.81 in a dataset of real requirements. Moreover, the approach finds 90% more issues that the experienced engineers found than the direct approach, and reports an average of 6 new issues per diagram.

***Index Terms*—UML, Sequence Diagrams, Large Language Models (LLMs), LLM-as-a-judge, Model Evaluation.**

I. Introduction

During the process of model-driven engineering [1]–[3], requirement engineers iteratively design and improve the system requirements, then develop diagrams based on these requirements. These diagrams facilitate the communication between stakeholders, they guide and provide a structure for the implementation, and reduce the maintenance cost of the system. Behavioral diagrams, e.g., activity diagrams, sequence diagrams, etc., are a type of diagram that detail the dynamic behavior of systems, such as the interactions between objects, state changes, and responses to external events.

Model diagrams are designed from textual requirements documents [4] by engineers, with or without the assistance of Artificial Intelligence (AI) tools [5]–[8]. As both error-prone manual labor [9], [10] and hallucination-prone [5], [11]–[13] AI agents are involved in the development of the diagrams, rigorous evaluation of their quality is needed.

Several techniques [14]–[19], by transforming the model into a language with formal semantics, have been proposed to verify if the diagram satisfies behavioral properties and object constraints, or check inconsistencies with other models of the same system using model checking [14], [15], theorem proving [17], or consistency checks [18]. However, these formal verification techniques do not evaluate the model against the textual requirements description and rely on the assumption that the diagram is already of high quality, i.e., the diagram completely and accurately captures the textual requirements' description. Some approaches evaluate a model against a reference written in a formal language [10], [20]–[26]. Yet, these approaches fail to evaluate alternative yet valid models, and a structured formal model may not be available in many real-world scenarios. As requirements are written in natural language and have no formal semantics, it is challenging to develop automated techniques that can evaluate the quality of the diagram against the requirements' description.

Large Language Models (LLMs) as a judge (LLM-as-a-Judge) was recently proposed for the evaluation of formal artifacts (e.g., code) against natural language [27]–[30] and has demonstrated reliable processing and reasoning capabilities for informal text. We propose an automated diagram evaluation technique that evaluates the accuracy and completeness of the diagram against free-style requirements texts by utilizing the capabilities of LLMs. However, directly evaluating the entire diagram against the requirements using LLMs does not work due to hallucinations [11] and failures to focus on the entire context [31]. We show in a preliminary study that the *holistic* approach results in an inadequate evaluation, i.e., fails to *focus* on all details in the diagram [31], and thus only reports 34% of issues that experienced engineers are able to report, and

We thank Ru Ji and Zohreh Aghababaeyan from Huawei Research Canada for their insightful discussions and valuable feedback, as well as the anonymous reviewers for their constructive comments and helpful suggestions.

*Work partially done during an internship at Huawei Research Canada.

979-8-3315-4910-7/25/$31.00 ©2025 IEEE
DOI 10.1109/MODELS67397.2025.00014

Authorized licensed use limited to: Institut Teknologi Sepuluh Nopember. Downloaded on September 29,2026 at 00:44:30 UTC from IEEE Xplore. Restrictions apply.

hallucinates non-existing issues [11].

We propose a more comprehensive, fine-grained, and multi-perspective technique to evaluate diagrams: *atomic checking*; we split the diagram and requirements into *atomic*, indivisible interactions and short, concrete requirements, respectively. We then compare the diagram with atomic requirements and each diagram-atom with the requirements. Thus, the LLM can *focus* on each part of the diagram and requirements, and has better chance of finding issues. Moreover, we propose a novel self-consistency approach to mitigate hallucinations by cross-checking the results from different checks, where the hallucinated issues originate from less reliable checks are removed if they conflict with the results from more reliable ones. This process leverages the difference of strengths between each type of check, while retaining the unique perspective of each check. Overall, we augment the holistic check with atomic checking, and combine the results from different checks through the self-consistency approach to reduce hallucinations.

We implement our approach in a Model Correctness evaluation Tool (MCeT). We evaluate our approach on a dataset of real, industrial requirements and AI-assisted human-generated sequence diagrams, along with diagram issues reported by experienced engineers [5]. We show that our approach is able to detect 65% of human-reported issues, which is 90% more than the issues detected by the solitary holistic approach, while achieving a high precision of 81%, and reporting an average of 6 issues not reported by experienced engineers.

To the best of our knowledge, MCeT is the first LLM-based approach to evaluate a behavioral diagram model against free-style requirements texts, detecting discrepancies between them, and reporting all issue explanations in natural language. Thus, closing the automated quality assessment gap from requirements to model diagrams, helping to produce high-quality artifacts for model-driven engineering activities.

Contributions. This paper makes the following contributions.
(1) We propose the first automated behavioral model evaluation approach to evaluate a behavioral diagram model against its free-style requirements textual description.
(2) We provide an implementation of our approach in a tool named MCeT and make it publicly available [32].
(3) We evaluate MCeT on a set of real requirements and sequence diagrams, and show that the tool has high precision and high human-reported issues recall. The implementation, prompts, and the evaluation dataset used by MCeT are available online [32].

II. Background

This section describes the key concepts used in the work.

Requirements. We focus on functional requirements, which describes essential functionalities and interactions of the system. Requirements text can be structured in several ways, some of the most common are the hierarchical fashion, i.e., starting with high-level requirements and breaking them down to sub-requirements, or grouped into use cases with preconditions, postconditions, and steps, or user stories which describe what the user wants to achieve, or in a free-style natural description. In this work, we do not assume any structure and can process all textual requirements' descriptions.

Sequence diagram. The sequence diagram shows the interaction between the *participants* of the system, e.g., users, services, and databases. Each participant has a *lifeline* representing their active period. Participants pass around *messages* in top to bottom order of the diagram, these messages carry around instructions and data. Messages may trigger *activation bars* which indicate active processing. Messages may be shown inside *combined fragments*, which represent more complex interactions, such as conditional or repeating paths. For example, an "alt" block fragment shows two alternative paths, and describes the condition for each path.

PlantUML. Sequence diagrams can be designed using tools such as PlantUML [33]. This tool provides a formal syntax for describing the diagram in a text file, and an engine that converts the syntax into a visual diagram. Thus, automatically processing a sequence diagram designed with PlantUML is as simple as parsing the syntax, converting it to an abstract syntax tree, and performing any further processing on the tree.

LLM-as-a-Judge. Large language models (LLMs) are generative models capable of producing natural language outputs given a textual input. Typically, both input and output texts of an LLM are split into sequences of *tokens*. With increased scale and complexity, LLMs have demonstrated capabilities in various *reasoning* tasks, such as mathematical problem-solving, code generation, question-answering, etc. [34].

Recently, using LLMs to evaluate the quality of inputs, known as *LLM-as-a-Judge*, has gained popularity [27]–[30]. Such evaluation methods have been applied to various tasks, including code generation, text summarization, translation, open-domain question answering, etc. [35], [36]. However, their use in model-driven engineering remains limited.

Typically, an LLM-as-a-Judge approach generates a numeric score representing the quality of the evaluated input [37], [38]. A significant drawback of this method is its lack of explainability, as it provides limited insight into how the score was determined. To address this limitation, alternative approaches [39], [40] leverage LLMs to explicitly identify and describe specific *issues* in the inputs, resulting in finer-grained and more interpretable evaluations. In this paper, we adopt this latter approach, proposing an LLM-based evaluation to detect and describe issues in sequence diagrams.

III. MCeT on an Example

We explain our approach on the example in Figure 1. The requirements describe that *Alice* interacts with a system that loads and shows the user data to her. The diagram covers most of the requirements; however, it has two problems, (1) the *Display data* message from *Bob* should go to *Alice* instead of *Cat*, and (2) the condition *[Exception]* and its associated message *Show error* are not part of the requirements.

Our approach aims at detecting and reporting these problems in the diagram to the user. To this end, MCeT uses the requirements as a reference in evaluating the quality of the diagram, but makes no assumptions about formality, structure,

Authorized licensed use limited to: Institut Teknologi Sepuluh Nopember. Downloaded on September 29,2026 at 00:44:30 UTC from IEEE Xplore. Restrictions apply.

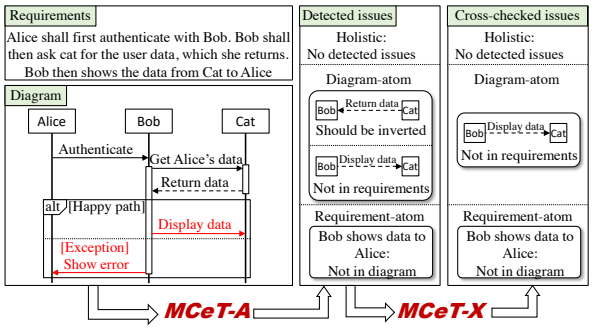


Fig. 1. A requirements text, its faulty associated diagram, and the detected issues at both stages of MCeT.

or style of the document, i.e., can handle free-style text, and evaluates that the diagram is both ① *Accurate*: The behavior described by the model is consistent with the requirements, and ② *Complete*: The model covers all requirements present in the text with sufficient detail. We borrow these definitions from prior work on manually evaluating sequence diagrams [5].

A straightforward approach to evaluate this diagram is the *Holistic* approach, where the entire diagram is evaluated against the requirements with one LLM invocation. We perform a pilot study to assess the effectiveness of the holistic approach in detecting issues. The full details of this study are presented in Section V and Section VI. The study shows that, on 16 diagrams with a total of 27 issues reported by humans, the holistic approach only detects 10 issues (37%). This result indicates that the holistic approach does not check every aspect of the diagram or requirements in detail.

Thus, we propose to augment the holistic approach with a more fine-grained approach; we decompose the holistic check into its two complementary checks that it implicitly does: checking that the diagram elements accurately reflect the requirements, and checking that all requirements are completely implement by the diagram. We split the diagram into *diagram-atoms*: indivisible interactions in the diagram, which we define as a message and its two involved participants, e.g., (Alice) – Authenticate → (Bob) is a diagram-atom. Prior work has proposed several other "atoms" that can be checked during the manual correctness checking of sequence diagrams [41], e.g., incorrect messages, participants, lifelines, conditions, etc. Our definition of a diagram-atom covers both messages and their participants, which are the main building blocks and the most frequent elements of a sequence diagram. MCeT then checks that the atom is accurately implemented within the diagram; the check evaluates the accuracy of diagram's elements, i.e., the message name, its type, the participants, the direction of the interaction, and the surrounding context of the atom which includes previous and following interactions, notes on the message, conditions, combined fragments, etc.

We also split the requirements into *requirement-atoms*: indivisible concrete requirements, which we define as a requirement that includes at most one action involving one or more participants, e.g., *Alice shall first authenticate with Bob* is a requirement-atom that involves the "authenticate" action only, and two participants, Alice and Bob. MCeT then checks that all elements and interactions mentioned in the requirement-atom are implemented in the diagram, i.e., evaluating the completeness of the diagram, and checking that the implementation is accurate by evaluating the accuracy of the related messages, participants, notes, combined fragments, and other diagram elements mentioned in the atom.

The input of MCeT is both the requirements text and a textual format of the diagram in a modeling language. MCeT works in two stages, the first of which is the **A**tomic checking stage, MCeT-A, which performs three types of checks: *Holistic check*: evaluates the entire diagram against the requirements, *Diagram-atom check*: evaluates each diagram-atom in the diagram against the requirements, and *Requirement-atom check*: evaluates the diagram against each atomic requirement.

The center part of Figure 1 shows an example output of the MCeT-A which is in the form of a list of issues found in the diagram, along with the localization of these issues. For example, MCeT-A reports that the requirement-atom *Bob shows data to Alice* is not implemented in the diagram, because in the diagram, *Bob* sends the *Display data* to *Cat* instead of Alice. MCeT-A may, however, hallucinate some non-existing issues such as (Bob) ← Return data – (Cat): *Should be inverted*, or miss some issues, such as (Alice) ← Show Error – (Bob) which should not be in the diagram.

To reduce such hallucinated issues, we develop a self-consistency step that employs an authority-based **cross**-check between the results of different checks in a second stage of the tool, MCeT-X. The cross-check identifies cases where a low-authority check reports an issue in the diagram but another, high-authority check marks relevant parts of the issue as correct; based on the contradiction, we deem the issue as a hallucination and remove it. For example, the issue (Bob) ← Return data – (Cat): *Should be inverted* was detected by the diagram-atom check, but no issues were found in the requirement-atom related to Cat returning data to Bob. Thus, MCeT-X eliminates that issue from the final output, which is shown on the right side of Figure 1. Next, we describe the different types of checks, discuss how the hierarchy of authority is decided for the cross-check, and show how both stages of MCeT work.

IV. Approach

The overview of MCeT is shown in Figure 2. MCeT takes two inputs, a requirements document and a sequence diagram, possibly generated by an AI assistant. In our current approach, we process sequence diagrams in PlantUML, however, the technique can be easily adapted to process any other modeling language. We first describe how the first stage of MCeT, MCeT-A, evaluates the sequence diagram given the requirements by describing each of its three checks, and then present the authority-based cross-check in the second stage, MCeT-X.

Authorized licensed use limited to: Institut Teknologi Sepuluh Nopember. Downloaded on September 29,2026 at 00:44:30 UTC from IEEE Xplore. Restrictions apply.

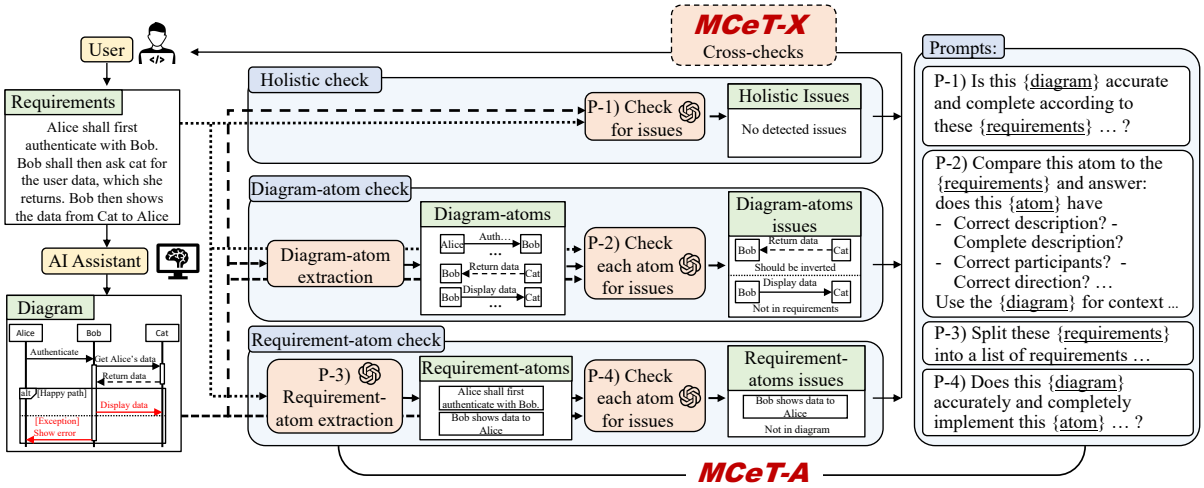


Fig. 2. Overview of MCeT.

A. Holistic check

The holistic check relies on one invocation of the underlying LLM with the prompt P-1, summarized in Figure 2. The prompt's inputs are the entire diagram, in PlantUML syntax, and the entire requirements text. The prompt provides few-shot examples [42] and asks the LLM to compare the diagram and requirements to each other, use chain-of-thought [43] to analyze the differences, and then find accuracy and completeness issues in the diagram.

To improve reliability, we utilize voting [44], [45] between several LLM responses for the same prompt. Specifically, after collecting `N` sets of accuracy and completeness issues from `N` LLM responses, we do one more LLM invocation, this time asking the LLM to keep issues that appear in `N/2` of the responses, and discard the rest. We use the language capability of LLMs for combining votes as the same issue could be phrased in different ways in each response.

B. Diagram-atoms check

The first step of this check is to extract atoms from the diagram. We simply parse the PlantUML file and extract the declaration of each message and its participants to form an atom. The atom is used as input to the prompt P-2, shown in Figure 2. The requirements are provided as input so that the accuracy and completeness of the atom are checked against the requirements. We provide the full diagram as input to the prompt as well to help the LLM understand the context of the atom. For example, the atom (Bob) ← Return data – (Cat) is only accurate when it follows the (Bot) – Get Alice's data → (Cat) atom in the diagram. The prompt instructs the LLM to understand such ordering information and other complex interactions from the diagram. Finally, the prompt asks questions on the accuracy and completeness of the atom's elements (message, participants, and the direction) and the atom's context. We also utilize voting in this check where we only keep issues appearing in `N/2` of the LLM responses.

C. Requirement-atoms check

This check leverage the reasoning capability of LLMs to split the requirements into requirement-atoms by an LLM invocation using prompt P-3. For example, the sentence "Bob shall then ask cat for the user data, which she returns" is split into "Bob shall ask cat for the user data", "Cat shall return the user data to Bob".

We then invoke the LLM with prompt P-4 which takes the diagram and requirement atom as inputs, and asks the LLM to check if the diagram implements all elements and interactions mentioned in the atom completely, and to check the accuracy of that implementation. We also utilize voting for this step.

D. Higher authority cross-check

As shown in Figure 1, the results from MCeT-A are prone to hallucinations. For example, (Bob) ← Return data – (Cat): *Should be inverted* is not a real issue. To mitigate these hallucinations, we propose to perform a self-consistency evaluation across the results of the three checks of MCeT-A. Typical self-consistency relies on majority voting between several LLM responses [44], However, we rely on the observation that each different type of check has its complexities, strengths, and weaknesses [46]. We propose to augment voting with a self-consistency evaluation that combines the complexity and strengths of different checks, while attenuating their weaknesses to further reduce hallucinations.

TABLE I
ANALYSIS OF ISSUES FROM 16 DIAGRAMS

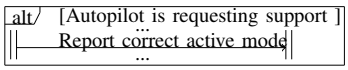
MCeT w/ `GPT-4o-mini`	Holistic issues	Diagram-atom issues	Requirement-atom issues
Total	25	75	161
True positives	18	30	142
Precision	0.72	0.4	0.88

Thus, we assess the strengths and weaknesses of each of MCeT-A's atomic checks; we evaluated the correctness of the issues detected in all three checks on 16 diagrams, we detail

Authorized licensed use limited to: Institut Teknologi Sepuluh Nopember. Downloaded on September 29,2026 at 00:44:30 UTC from IEEE Xplore. Restrictions apply.

the methodology for this assessment in Section V. Table I shows the result of this assessment. The "Total" row shows the total number of issues in each of the outputs of the three checks, the "True positives" shows the number of output issues that we deem as real issues, and the "Precision" is the ratio of true positives out of the total detected issues. The table shows that the precision of the diagram-atom issues is low, only 0.4, compared to that of the requirement-atom issues.

This phenomenon is because a sequence diagram message is always a part of a bigger context, the following example shows a message that only executes under a condition:



The task of an LLM is to assess whether this message is accurate, this assessment is context-sensitive. This is because the reported issue can be related to elements of the atom or the surrounding context, as discussed in Section III. Thus, MCeT-A reports the following false positive issue for the above message related to its context:

> *... lacks the necessary condition that the autopilot is in a state requesting support, which is essential ...*

On the other hand, the requirement-atom check does not exhibit this problem, because the task of the LLM is to assess correctness of the diagram, not the requirements. This is a simpler task for the LLM than the diagram-atom check as the check is context-free; the LLM has to only check whether all elements of the requirement atom are implemented accurately in the diagram, the LLM does not need to extract any context for a requirement-atom to check that the diagram accurately and completely adheres to that atom.

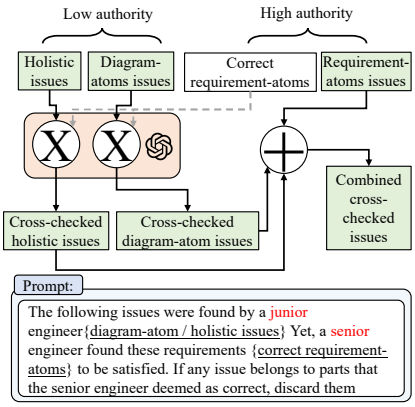


Fig. 3. Cross-checking and combining results of different checks.

Indeed, the requirement-atom check detects 6X the number of issues detected by the holistic approach, and 2X the number of issues detected by the diagram-atom approach. Based on this observation, we propose an approach to leverage the higher precision and recall of the requirement-atom check to automatically filter out false positive issues from the lower precision and recall holistic and diagram-atom checks. Figure 3 shows an overview of this approach, which we name *Higher authority cross-check* and adopt in the second stage of our approach MCeT-X.

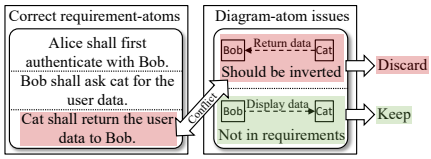


Fig. 4. Cross-checking results of the motivating example.

First, we identify *Correct Requirement-atoms*, which are requirement-atoms that MCeT-A did not report any issues with. Then, we perform two independent cross-checks, using two LLM invocations, one for the holistic issues, and the one for the diagram-atom issues. The cross-check uses the prompt shown in Figure 3. To relay the hierarchy of authority between the issues under check and the correct requirement-atoms, the prompt refers to the issues under check as issues found by a *junior engineer*, while the correct requirement-atoms as requirements found correct by a *senior (higher authority) engineer*. The LLM is then instructed to find and discard low-authority issues that conflict with correct requirements.

Figure 4 shows the result of applying the cross-check technique on the example from Figure 1. The correct requirement-atoms are those atoms without issues, i.e., all atoms except for *Bob shows data to Alice*, which is reported as a requirement-atom issue in Figure 1. All diagram-atom issues reported in Figure 1 are cross-checked against the correct requirement-atoms. The LLM finds that the issue: (Bob) ← Return data – (Cat): *Should be inverted*, is in conflict with the correct requirement-atom *Cat shall return the user data to Bob*. As the conflicting requirement-atom is deemed correct by a higher authority, we consider the diagram-atom issue as a hallucination and discard it. We keep diagram atom issues that are not in conflict with any correct requirement-atoms. For example, the issue (Bob) – Display data → (Cat): *Not in requirements* is correctly kept.

However, not all diagram-atom issues can be matched with a conflicting requirement-atom issue. This is because a requirement-atom issue is related to elements of the requirement-atom, which is related to a self-contained part of the diagram, e.g., all user authentication messages. However, diagram-atom issues can be related to either elements of the atom, or the context, which could contain several messages, i.e., the previous and next messages, which could span several requirement-atoms. Thus, not every diagram-atom issue can be matched with one conflicting requirement-atom, cross-checking does not remove these issues that span several requirement-atoms, and retains the unique-perspective of the diagram-atom context issues.

Finally, the kept issues from both the holistic issues and diagram-atom issues, called *Cross-checked holistic issues* and *Cross-checked diagram-atom issues* in Figure 3, are combined with the *Requirement-atom issues* to produce the *Combined cross-checked issues* list, the final output of MCeT-X.

Authorized licensed use limited to: Institut Teknologi Sepuluh Nopember. Downloaded on September 29,2026 at 00:44:30 UTC from IEEE Xplore. Restrictions apply.

V. Evaluation

To evaluate MCeT, we ask the following research questions:
RQ1 (Precision): How effective is MCeT in detecting real issues versus a baseline holistic approach?
RQ2 (Human-detected issues): How does MCeT compare with the holistic approach in detecting human-reported issues?
RQ3 (LLM trade-offs): What are the trade-offs of using different reasoning and non-reasoning LLMs in MCeT?

A. Setup

Dataset. We evaluate our approach on a set of requirements documents and their scored sequence diagram models which we obtain from Ferrari et al. [5] and use as our benchmark. We name this benchmark **FBench**. The 28 requirements in this benchmark were collected from various specifications of real industry projects, including the Lockheed Martin cyber-physical domain requirements [47], the PURE multi-domains and multi-format requirements [48], and a dataset of user stories [49]. Two authors of Ferrari et al. [5] introduced a set of variants by incrementally introducing requirement smells, such as ambiguity, inconsistency, and incompleteness. These smells introduce issues in their corresponding generated diagrams, which we aim to detect with MCeT. Our selection of this dataset thus ensures that we evaluate MCeT on requirements of practical, non-ideal quality.

To generate sequence diagram models, Ferrari et al. [5] used an LLM (ChatGPT's `GPT3.5`) to generate a diagram for each variant. Each generated diagram was scored on a zero-to-five scale by one of two researchers, who are also experienced software engineers, according to five metrics. Two of the metrics are *Correctness* and *Completeness*, defined the same as our definition for accuracy and completeness from Section III, respectively. We focus on these two metrics as MCeT is designed to find accuracy and completeness issues only. Whenever the score is not a perfect five, the researchers identified the issues in the diagram that affected the score, we use these issues as a ground truth of issues found in the diagram by experienced engineers. More details on the steps taken to ensure the reliability of the ground truth are provided in Ferrari et al.'s paper [5]. The total number of variants is 87, we discarded cases where the model generated the wrong type of diagram, e.g., a class diagram, or the researchers did not provide reasons for the score deduction. The remaining number of variants is 76, which we use to evaluate MCeT.
Subjects. We evaluate the outputs of each of the MCeT checks independently to gain better understanding of how each check contributes to the performance of the tool. Specifically, we evaluate the MCeT-A output from the holistic check, diagram-atom check, and requirement-atom check. We also evaluate the MCeT-X outputs for the holistic results and diagram-atom results which are cross-checked against the requirement-atom issues. Finally, we also evaluate the combined MCeT-A output, and the combined MCeT-X output. We use the direct holistic check results as a comparison baseline.
LLMs and MCeT Configuration. We use `GPT-4o-mini` within MCeT to evaluate the diagrams in FBench. `GPT-4o-mini` is selected as it is a lightweight, cost-efficient, and fast LLM with good performance [50]. We also repeat the evaluation for a subset of FBench using `GPT-4o`, `DeepSeek-v3`, `DeepSeek-R1`. `GPT-4o` [51] is a larger, more versatile, and more intelligent version of `GPT-4o-mini`. `DeepSeek-v3` [52] is an open-source, large model that employs the Mixture of Experts (MoE) technique that "activates" specialized parts of the trained network depending on the input. `DeepSeek-R1` [53] is an open-source LLM that is trained to employ reasoning, achieves significantly high performance in math, coding, and scientific tasks. We show through our evaluation that our technique is robust to changes in the underlying LLM, giving good accuracy even on a lighter-weight LLM like `GPT-4o-mini`.

All LLMs are configured with a temperature and top-p of 0.7 and 1, respectively. The number of votes is set to five.

B. Methodology

To assess the quality of the issues that MCeT reports, which is required to answer all of our research questions, we rely on the human judgement of two of the authors of this paper. Both authors are software engineers with 4 and 6 years of experience. One of the authors participated in teaching a software engineering course for three offerings where he graded sequence diagrams submitted by students. The other participated in teaching and grading for an algorithms course.

To divide the work of judging the issues among the two authors, we first assessed that both authors have similar judgement such that we can reliably combine their results. To perform this assessment, we selected the first 20% of the diagrams (according to the order in the FBench paper [5]) as a small dataset to assess the similarity on. Then, we run MCeT on the selected diagrams, then both authors independently judged the correctness of all detected issues, i.e., each author gives a "Yes" or "No" rating for the correctness of each issue, independent of the other author's rate. We then measured the inter-rater reliability using the Cohen's kappa statistic [54], which results in 0.79, indicating a substantial agreement, close to 0.8 which indicates almost perfect agreement. Afterwards, both authors discussed the rating of issues for which they disagreed. Both authors settled the disagreements and decided on a common approach of judging any future issues similar to the disputed issues. The results after settling the disagreements are presented and discussed in Section IV-D.

To answer RQ1 and RQ2, we split the remaining MCeT-detected issues among both authors who proceeded to judge the correctness of each issue. We record the following metrics: ① *Total*: the total number of MCeT-detected issues, ② *True positives*: the number of correct MCeT-detected according to the judging author, ③ *False positives*: the remaining issues out of "Total" that are not "True positives", ④ *Precision*: ratio of "True positives" out of the "Total" number of issues, ④ FBench *recall*: the number of human-detected issues in FBench that are equivalent to at least one MCeT-detected issue. We define equivalent issues as issues that describe the same root cause of the problem in the diagram, even if they

Authorized licensed use limited to: Institut Teknologi Sepuluh Nopember. Downloaded on September 29,2026 at 00:44:30 UTC from IEEE Xplore. Restrictions apply.

TABLE II
RESULTS OF APPLYING MCET ON THE FULL FBENCH USING GPT-4O-MINI AS THE UNDERLYING MODEL

Metrics (gpt-4o-mini)	Holistic check (Baseline)		Diagram-Atom check		Req.-Atom check (MCeT-A / MCeT-X)	Combined checks	
	MCeT-A	MCeT-X	MCeT-A	MCeT-X		MCeT-A	MCeT-X
Total	134	69	505	201	885	1524	1155
True positives	78	43	254	131	764	1096	938
False positives	56	26	251	70	121	428	217
Precision	0.58	0.62	0.5	0.65	0.86	0.72	0.81
FBENCH recall	46 (34.1%)	27 (20%)	43 (31.9%)	32 (23.7%)	80 (59.3%)	92 (68.1%)	88 (65.2%)
New true issues	27	13	138	56	322	487	391

have different levels of details. For example, Figure 5 shows a human-detected issue (top) and an equivalent MCeT-detected issue (bottom). While the human issue is more generic and addresses the missing conditions in the entire diagram, the MCeT issue focuses on missing conditions in one message, which is a symptom of the same root issue the human reported. The last metric ⑤ *New true issues* is the number of "True positives" that are not equivalent to any issue reported by the FBENCH authors, i.e., these are real issues MCeT found, but FBENCH authors did not find or did not report.

Human: the diagram … overlooks some specific conditions that are particularly hard to interpret also for a human.

MCeT: The … Request Movement Authority message correctly reflects the requirement for requesting movement authority but lacks the context of conditions under which the movement …

Fig. 5. An equivalent pair of a human and an MCeT-detected issue. The common root issue is missing conditions.

To answer RQ3, we re-run MCeT configured with `GPT-4o`, `DeepSeek-v3`, and `DeepSeek-R1` on 10% of the dataset (8 diagrams) due to the extensive time and effort required to manually evaluate issues: we evaluated a total of 1524 issues for RQ1 and RQ2, and 347 issues for RQ3. To ensure we evaluate LLM trade-offs on a wide variety of diagrams, we compute the precision of MCeT for each case in FBENCH using `GPT-4o-mini`, we split the 0-1 precision range into 8 equal segments, and randomly select one diagram from each segment. Thus, we ensure that we repeat our evaluation for different LLMs on a range of diagrams for which `GPT-4o-mini`-MCeT performed both well and poorly.

We compute the metrics used in answering RQ1 and RQ2 with this new configuration. We also measure *Average K tokens / diagrams*: the average number of tokens processed by the LLM invocations within MCeT per diagram, and *Average minutes per diagram*: the average execution time of MCeT in minutes per diagram. We then identify the trade-off of the precision and the number of human-detected issues reported by MCeT with the execution time and number of tokens.

VI. RESULTS

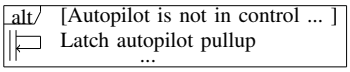
Table II shows the result of applying MCeT on all pairs of sequence diagrams and requirements in FBENCH where MCeT uses `GPT-4o-mini` as the underlying LLM. The rows of the table are the metrics defined in Section V-B. The "Holistic check (Baseline)" are the issues that are both output directly from the holistic check (MCeT-A column) and from cross-checking (MCeT-X column) against the requirement-atom check results. The "Diagram-atom check" results also include both MCeT-A and MCeT-X issues, while the issues in the "Req.-Atom check" column are the same in MCeT-A and MCeT-X since the requirement-atom MCeT-A results are the higher authority used as reference for cross-checking. The "Combined checks" combines all the direct outputs from the holistic, diagram-atom, and requirement-atom checks' results (MCeT-A column), and combines the cross-checked results from the holistic and diagram-atom checks with the direct results of the requirement-atom check (MCeT-X column). We use this table to answer both RQ1 and RQ2.

A. RQ1 (Precision)

As shown in Table II, the baseline holistic check's precision (MCeT-A) is only 0.58, potentially because the holistic check compares the entire diagram with the entire requirements. When the diagram and requirements are long, the underlying LLM often overlooks details in the prompt which includes both the diagram and the requirements, a phenomenon common for LLMs [31], and thus falsely reports that the diagram is not complete. For example, in the `finite_state_machine.v0` case [32], the number of tokens in the requirements and diagram is 953, 2.3X times the average number of tokens of 406 for a requirement-diagram pair in FBENCH. The holistic check in this case reports that

The diagram omits critical details ... including the need to latch a pullup when the pilot is not in control ...

Even though the diagram does include the message:



The MCeT-A diagram-atom check also suffers from low precision (0.5) due to underlying LLM failing to extract the context, as observed and reported in the MCeT-A assessment from Section IV-D. The requirement-atom check achieves a significantly better precision of 0.86, also correlating with MCeT-A assessment results.

Our combined atomic and holistic approach achieves a 0.72 precision which is improved to 0.81 with cross-checking, indicating that cross-checking eliminates false positive issues. For example, the following issue is eliminated:

The message 'Pilot takes control' does not specify whether the pilot is in standby, which is critical for ...

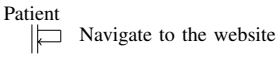
which conflicts with the following requirement-atom that was deemed correctly implemented by the tool:

Authorized licensed use limited to: Institut Teknologi Sepuluh Nopember. Downloaded on September 29,2026 at 00:44:30 UTC from IEEE Xplore. Restrictions apply.

The autopilot shall change states from TRANSITION to STANDBY when the pilot is in control (standby).

Since the requirement-atom related to the pilot being in control in the standby mode is deemed as correctly implemented, the related diagram-atom issue is correctly discarded by MCeT-X.

Cross-checking also removes some real issues, for example, for the following message in `g02-uc-cm-req.v0`:



MCeT-A reports this issue correctly:

The message ... fails to specify that it is an interaction with the pharmacy website rather than an internal action.

i.e., the message should go from *Patient* to *Website* instead of back to *Patient*. However, the requirement-atom:

The patient navigates to the website on his computer

is deemed as correct, because the LLM finds that the *Navigate to the website* is in the diagram, but fails to detect the issue with the other participant of this message, the website. Thus, the correctly identified diagram-atom is filtered out. However, we achieve high precision because cross-checking successfully reduces the number of false positive issues in the combined MCeT-A results from 428 to 217 in the combined MCeT-X results, eliminating 211 false positives, while only reducing true positives by 158 issues from 1096 to 938. Thus, MCeT-A is beneficial when higher true positives are desired by the user (e.g., in safety critical systems), while MCeT-X is useful in exploratory projects, when the user desires less false positives.

We observe that some false positives not filtered out by MCeT-X are caused by vague requirements. For example, in `1.autopilot.v0` [5], the requirement indicates that the "autopilot should only be engaged when the pilot selects it", and also indicates "autopilot shall engage when pilot selects the autopilot engage switch". Both sentences refer to the same requirement and thus are correctly represented once in the diagram. Yet, the first sentence does not mention "engage switch", and thus, MCeT-X assumes that it is not covered by the diagram. Thus, MCeT-X false positives can help the user spot problems in the requirements.

Answer to RQ1: MCeT achieves a precision of 0.81 by augmenting the holistic check with the atomic and cross-checking approaches, improving over the 0.58 achieved by the baseline holistic-only approach.

B. RQ2 (Human-detected issues)

The "FBENCH recall" row of Table II shows that the combined results of both MCeT-A and MCeT-X significantly outperforms the baseline holistic check, the recall increased from 34.1% to 68.1%, and 65.2% for MCeT-A and MCeT-X, respectively, i.e., MCeT has almost twice the recall of the solitary baseline holistic check. The atomic check results also add depth compared to the holistic results, as it identify more fine-grained and detailed issues. Figure 6 shows an FBENCH issue and an issue detected by MCeT from the use case `non_linear_guidance.v0` [32]. In this example, the issue reported in FBENCH is too broad and fails to pinpoint the specific problem with the diagram, while MCeT-reported issues identify the concrete problematic requirement-atom (NLGuidance), and indicate the concrete problem with the atom (missing actions related to the port-side).

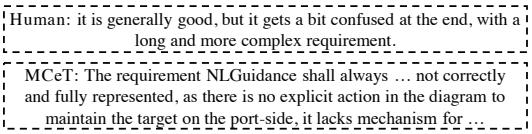


Fig. 6. An example of an FBENCH issue and a matching, yet more concrete, issue reported by MCeT.

We observed that the recall of the MCeT-A diagram-atom check is only 31.9%, potentially because it is unable to identify issues where entire diagram-atoms are missing. Since the diagram-atom is missing from the diagram, it cannot be checked for accuracy or completeness. On the other hand, the requirement-atom check can find if the requirement-atom is not fully implemented, i.e., the diagram misses essential diagram-atoms. For example, the bottom two issues from Figure 7 show a diagram-atom issue and a requirement-atom issue, the requirement-atom issue can find that a whole process is missing from the diagram (the assessment of the system's damage), while the diagram-atom issue focuses on *specifics* of existing diagram-atoms. Thus, the diagram-atom issues provide a different perspective, despite its lower recall.

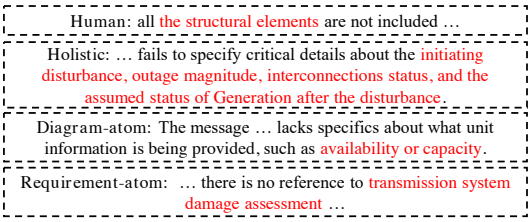


Fig. 7. Several issues related to missing structural elements, yet each check indicates a different missing element.

MCeT also identifies new issues that are not reported by the FBENCH authors. MCeT-A's diagram-atom and requirement-atom checks identifies 138 and 322 issues overlooked by humans, respectively. This is nearly 5X and 12X the number of new issues detected by the baseline holistic check. In the combined MCeT-A results, the total number of new issues discovered rises to 487, which is 18X the issues identified through the direct holistic check. Even after cross-checking, which filters out some true positive issues, the total number of issues identified in MCeT-A is 391, which is still 14X the issues found through the baseline holistic check. On average, the direct combined checks is able to identify 6 new issues per diagram, 5 issues after cross-checking.

The total number of true issues for MCeT-A and MCeT-X is 1,094 and 938, which is 14X and 12X the 78 true issues detected by the baseline approach, respectively. Moreover, combining issues from the three types of checks gives different perspectives to the same problematic part of the diagram.

Authorized licensed use limited to: Institut Teknologi Sepuluh Nopember. Downloaded on September 29,2026 at 00:44:30 UTC from IEEE Xplore. Restrictions apply.

TABLE III
TRADE-OFFS FOR USING DIFFERENT LLMS WITHIN MCET

Metrics (10% of dataset)	GPT-4o-mini			GPT-4o			DeepSeek-v3			DeepSeek-R1		
	Baseline	MCeT-A	MCeT-X	Baseline	MCeT-A	MCeT-X	Baseline	MCeT-A	MCeT-X	Baseline	MCeT-A	MCeT-X
Total	14	97	75	12	75	64	16	70	52	24	105	65
True positives	5	51	43	10	63	54	7	45	37	17	87	57
False positives	9	46	32	2	12	10	9	25	15	7	18	8
Precision	0.36	0.53	0.57	0.83	0.84	0.84	0.44	0.64	0.71	0.71	0.83	0.88
FBENCH recall	3 (21.4%)	10 (71.4%)	10 (71.4%)	4 (28.6%)	12 (85.7%)	10 (71.4%)	5 (35.7%)	10 (71.4%)	7 (50%)	7 (50%)	12 (85.7%)	10 (71.4%)
New true issues	2	25	19	5	28	23	1	16	15	7	39	25
Avg. $\frac{\text{K tokens}}{\text{diagram}}$	12	70.8	80.5	13.5	78	86.7	18.1	109.2	124.9	31.1	177.5	236.2
Avg. $\frac{\text{minutes}}{\text{diagram}}$	0.6	4.1	4.5	0.5	4	4.5	1.5	9.3	10.3	8	55.4	77.4

Figure 7 shows a human-reported issue that indicates that all structural elements in the diagram are missing. The holistic check mentions several missing elements, but not all of them. The diagram-atom level mentions another missing structural element, and the requirement-atom level mentions yet another missing element. Combining the three checks together brings the evaluation feedback's comprehensiveness close to that of the human, while also providing concrete details.

Answer to RQ2: MCeT doubles the FBENCH recall, compared to the baseline holistic approach, and identifies 391 new issues compared to human experts, 14x the number of new issues detected by the baseline approach. MCeT's issues provides more concrete, detailed, and multi-perspective insights into the causes of the issues.

C. RQ3 (LLMs trade-offs)

Table III shows the result of applying MCeT on 8 random diagrams, which is 10% of the FBENCH using reasoning and non-reasoning LLMs. The metrics of the table are the same as the metrics of Table II, in addition to two metrics: the "Avg. K tokens / diagram" and the "Avg. minutes / diagram". Each three consecutive columns are the results for one LLM, e.g., columns 1-3 are the results when MCeT uses `GPT-4o-mini` as the underlying LLM. The columns correspond to the holistic check output, combined direct output, and cross-checked combined output, respectively.

The precision row in Table III shows that the atomic approach *consistently improves* upon the holistic only approach; the lighter-weight LLMs benefit the most: `GPT-4o-mini`'s and `DeepSeek-v3`'s precision improved from 0.36 to 0.57 and from 0.44 to 0.71, respectively. Even the more sophisticated LLM `GPT-4o`'s precision improved from 0.83 to 0.84. The reasoning LLM `DeepSeek-R1`'s precision also improved from 0.71 to 0.88.

Our approach improves the recall of FBENCH issues from the holistic baseline for the light-weight `GPT-4o-mini` LLM, and significantly improves the recall of the more-sophisticated `GPT-4o` and the reasoning `DeepSeek-R1` LLMs; the recall is improved from 28.6% to 71.4% and from 50% to 71.4%, respectively. Note that `DeepSeek-R1` starts with a higher holistic recall (50%) than the other LLMs. This is because the LLM's reasoning attempts a fine-grained evaluation, similar to the MCeT approach, for example:

> *... First, I'll go through each requirement and check how it's represented in the diagram ...*

Here, the LLM does a less structured version of our requirement-atom checking, however, it does not focus on each and every atom, it does not combine the analysis from both requirement-atom and diagram-atom perspectives, and does not have information about higher authority results to perform our cross-check. Thus, even the reasoning LLM benefit from MCeT's approach, both in precision and recall. Only `DeepSeek-v3` has a drop in recall (from 71.4% to 50%), however, `DeepSeek-v3` cross-checked results still outperform the baseline holistic precision and recall.

`DeepSeek-R1` and `GPT-4o` achieve the best precision and recall, but cost more tokens and time per diagram than the lighter-weight `GPT-4o-mini`. The best overall precision is achieved by `DeepSeek-R1` cross-checked result (0.88 precision) but consumes 2.7X the tokens and 17.2X the time that the runner-up `GPT-4o` (0.84 precision) takes. `DeepSeek-R1` MCeT-X also consumes 2.9X the tokens 17.2X the time than `GPT-4o-mini` while achieving the same recall. Thus, MCeT enables non-reasoning LLMs to achieve comparable performance to reasoning LLMs at a fraction of the cost.

Moreover, the light-weight `GPT-4o-mini`'s MCeT-X recall (71.4%) is even higher than `DeepSeek-R1`'s holistic recall (50%), while `GPT-4o`'s MCeT-X precision and recall (0.84, 71.4%) are both higher than `DeepSeek-R1`'s holistic precision and recall (0.71, 50%). Both `GPT-4o-mini` and `GPT-4o` MCeT-X require only 4.5 minutes, 0.56X the time that `DeepSeek-R1` requires for the holistic evaluation. Thus, MCeT-X meets or surpasses a reasoning-LLM based baseline holistic-only approach with a fraction of the cost.

Answer to RQ3: `DeepSeek-R1`-MCeT achieves the best performance (precision and recall), but requires 2.7X the tokens and 17.2X the time than `GPT-4o`-MCeT which achieves comparable performance. MCeT moreover enables `GPT-4o-mini` and `GPT-4o` to surpass `DeepSeek-R1` baseline holistic performance with 0.56X execution time.

VII. THREATS TO VALIDITY

Internal Validity. The subjective judgments from the two authors influences the evaluation of MCeT. To mitigate the potential bias, we computed Cohen's Kappa coefficient [54] between the two authors, which indicated substantial agreement. Both authors discussed and standardized the criteria for evaluating issues. Moreover, results for RQ3 may not generalize for the entire dataset. To mitigate this issue, we

Authorized licensed use limited to: Institut Teknologi Sepuluh Nopember. Downloaded on September 29,2026 at 00:44:30 UTC from IEEE Xplore. Restrictions apply.

selected 8 diagrams that span the entire precision spectrum in RQ1, i.e., we selected diagrams for which MCeT performed both well and poorly on the full dataset, to ensure that the RQ3 results are more likely to be reflective of the entire dataset.

External Validity. LLMs are probabilistic in nature, it is possible to get different issues each time MCeT is run on a use case. To address this consistency issue, we implemented a voting mechanism, combining the majority response from the answers. Finally, our approach may not generalize to other types of behavioral models, we aim to expand our study into other types of behavioral models as part of our future work.

VIII. Related Work

LLM-as-a-Judge. With the rise of LLMs, their use in replacing labor-intensive evaluation has attracted increasing interest [27], [28], [35], [36], with most approaches distinguishable by LLM's role in the evaluation process. A common category of methods extracts generation probabilities from the LLM as evaluation scores [30], [55], [56]. For example, GPT-Score [30] computes the conditional probability of the solution given the task description as the score. Another class of approaches prompts LLMs to directly output evaluation scores using a predefined grading scheme [37], [38]. This direct scoring strategy also enables the use of multiple LLMs in a multi-agent evaluation system [57], [58]. In the context of software engineering, Wang et al. [59] systematically evaluated the performance of LLM-as-a-Judge on various coding tasks.

A related line of work involves using LLMs for self-reflection to enhance generation quality [39], [40]. Methods such as Reflexion [60] and Self-Refine [61] aim to identify errors in current LLM outputs and provide feedback for subsequent generations, either to avoid or correct these mistakes.

MCeT leverages the LLM-as-a-Judge paradigm to detect issues in a sequence diagram by comparing it with the original textual requirements' description. Moreover, MCeT uses a finer-grained evaluations approach by breaking down the evaluation into different checks. Since each check focuses on different aspects of model quality, we further propose a *corss-check* strategy that combines strength of different checks to mitigate hallucinations from LLMs.

Automated model evaluation. Many approaches target automatically evaluating the quality of models, typically in an educational setting where an instructor-created *reference model* is available [9]. When a reference model is utilized, automated evaluation methods generally compare candidate models against this reference, identifying differences through rule-based or graph-matching approaches tailored specifically for UML model comparison [10]. Such comparison tool usually built around popular modeling frameworks such as the Eclipse Modeling Framework (EMF) [20], [21] and TouchCore [22], [23]. Based on identified mismatches, errors can be systematically extracted [24]. More recently, embedding-based techniques have complemented traditional graph-matching by capturing semantic similarities between model elements, further improving evaluation performance [25], [26].

However, a critical limitation of reference-based evaluation approaches is that they fail to consider alternative yet valid models. Furthermore, in many real-world scenarios, a suitable reference model may not even be available. Consequently, alternative methods aim to identify common modeling mistakes independently of reference solutions, typically employing rule-based heuristics [62] or machine learning classifiers [63], [64]. For instance, Boubekeur et al. [63] extracted heuristic rules based on a provided marking scheme to train classifiers predicting the quality of domain models.

Compared to existing methods, by utilizing LLMs, MCeT can directly compare candidate models against the text requirement descriptions, allowing the extraction of modeling errors without relying on explicit rules or training data. Although this paper primarily focuses on evaluating the sound foundation of models against the requirement, MCeT can be effectively combined with complementary approaches to identify and correct broader issues related to modeling practices.

LLM and MDE. In the context of model-driven engineering (MDE), the application of LLMs has become increasingly popular in modeling tasks, including model generation, completion, transformation, and the development of tools for domain-specific languages (DSLs) [65]. For model generation, LLMs are used to convert textual descriptions into complete model artifacts, such as domain models [66], [67], sequence diagrams [5], [8], and goal models [6]. In some cases, LLMs assist human modelers during the modeling process, e.g., in model completion [68], [69] and in model transformation [70]. Furthermore, LLMs can be utilized to translate natural language into DSL queries, such as VQL [71] and OCL [72].

Despite the promising capabilities of LLMs to generate model diagrams directly from textual input, the generated results often suffer from hallucinations, leading to misalignment with the original descriptions [66]. In this paper, we propose MCeT, an approach for automatically detecting such misalignments as issues. MCeT can serve as an automated evaluation mechanism for LLM-generated models or provide feedback for manual or automated correction of these issues.

IX. Conclusions

This paper presents a behavioral model evaluation approach, implemented in the tool MCeT, which evaluates a sequence diagram against the requirements and identifies and reports potential issues. MCeT leverages the language reasoning capabilities of LLMs and enhances them with fine-grained, multi-perspective, and self-consistency evaluation. To the best of our knowledge, MCeT is the first approach to perform fully automated evaluation of a behavioral model against its requirements. We evaluated MCeT on real requirements and their corresponding sequence diagrams and showed that it achieves high precision and human-like issue detection capabilities. We plan to investigate preventing the cross-checking approach from eliminating real issues, and generalizing our approach to more types of models in our future work.

Authorized licensed use limited to: Institut Teknologi Sepuluh Nopember. Downloaded on September 29,2026 at 00:44:30 UTC from IEEE Xplore. Restrictions apply.

REFERENCES

[1] D. Schmidt, "Model-Driven Engineering," *Computer*, vol. 39, no. 2, pp. 25–31, 2006.

[2] S. Kent, "Model Driven Engineering," in *Integrated Formal Methods (IFM)*, 2002, pp. 286–298.

[3] J. Hutchinson, M. Rouncefield, and J. Whittle, "Model-Driven Engineering Practices in Industry," in *Proc. of the International Conference on Software Engineering (ICSE)*, 2011, pp. 633–642.

[4] B. Nuseibeh and S. Easterbrook, "Requirements Engineering: A Roadmap," in *Proc. of the Conference on The Future of Software Engineering (ICSE)*, 2000, pp. 35–46.

[5] A. Ferrari, S. Abualhaijal, and C. Arora, "Model Generation with LLMs: From Requirements to UML Sequence Diagrams," in *Proc. of International Requirements Engineering Conference Workshops (REW)*, 2024, pp. 291–300.

[6] B. Chen, K. Chen, S. Hassani, Y. Yang, D. Amyot, L. Lessard, G. Mussbacher, M. Sabetzadeh, and D. Varró, "On the Use of GPT-4 for Creating Goal Models: An Exploratory Study," in *2023 IEEE 31st International Requirements Engineering Conference Workshops (REW)*. IEEE, 2023, pp. 262–271.

[7] J. Cámara, J. Troya, L. Burgueño, and A. Vallecillo, "On the assessment of generative AI in modeling tasks: an experience report with ChatGPT and UML," *Software and Systems Modeling*, vol. 22, no. 3, pp. 781–793, 2023.

[8] M. Jahan, M. M. Hassan, R. Golpayegani, G. Ranjbaran, C. Roy, B. Roy, and K. Schneider, "Automated Derivation of UML Sequence Diagrams from User Stories: Unleashing the Power of Generative AI vs. a Rule-Based Approach," in *Proceedings of the ACM/IEEE 27th International Conference on Model Driven Engineering Languages and Systems*, 2024, pp. 138–148.

[9] W. Bian, O. Alam, and J. Kienzle, "Is Automated Grading of Models Effective? Assessing Automated Grading of Class Diagrams," in *Proceedings of the 23rd ACM/IEEE International Conference on Model Driven Engineering Languages and Systems*, 2020, pp. 365–376.

[10] C. Tselonis, J. Sargeant, and M. McGee Wood, "Diagram Matching for Human-Computer Collaborative Assessment," 2005.

[11] L. Huang, W. Yu, W. Ma, W. Zhong, Z. Feng, H. Wang, Q. Chen, W. Peng, X. Feng, B. Qin, and T. Liu, "A Survey on Hallucination in Large Language Models: Principles, Taxonomy, Challenges, and Open Questions," *ACM Transactions on Information Systems*, vol. 43, no. 2, 2025.

[12] Y. Zhang, Y. Li, L. Cui, D. Cai, L. Liu, T. Fu, X. Huang, E. Zhao, Y. Zhang, Y. Chen, L. Wang, A. T. Luu, W. Bi, F. Shi, and S. Shi, "Siren's Song in the AI Ocean: A Survey on Hallucination in Large Language Models," 2023. [Online]. Available: https://arxiv.org/abs/2309.01219

[13] S. M. T. I. Tonmoy, S. M. M. Zaman, V. Jain, A. Rani, V. Rawte, A. Chadha, and A. Das, "A Comprehensive Survey of Hallucination Mitigation Techniques in Large Language Models," 2024. [Online]. Available: https://arxiv.org/abs/2401.01313

[14] J. Lilius and I. P. Paltor, "vUML: a tool for verifying UML models," in *Proc. of the International Conference on Automated Software Engineering (ASE)*, 1999, pp. 255–258.

[15] D. Latella, I. Majzik, and M. Massink, "Automatic Verification of a Behavioural Subset of UML Statechart Diagrams Using the SPIN Model-checker," *Formal Aspects of Computing (FAC)*, vol. 11, pp. 637–664, 1999.

[16] J. Jürjens, "Formal Semantics for Interacting UML Subsystems," in *Proc. of the International Conference on Formal Methods for Open Object-Based Distributed Systems (FMOODS)*, 2002, pp. 29–43.

[17] H. Ledang and J. Souquieres, "Contributions for Modelling UML State-Charts in B," pp. 109–127, 2002.

[18] A. Knapp and T. Mossakowski, "UML Interactions Meet State Machines - An Institutional Approach," in *Proc. of Conference on Algebra and Coalgebra in Computer Science (CALCO)*, 2017, pp. 15:1–15:15.

[19] F. Jouault, V. Besnard, T. L. Calvar, C. Teodorov, M. Brun, and J. Delatour, "Designing, Animating, and Verifying Partial UML Models," in *Proc. of the International Conference on Model Driven Engineering Languages and Systems (MODELS)*, 2020, pp. 211–217.

[20] Eclipse Foundation, "Eclipse EMF Diff/Merge," https://projects.eclipse.org/projects/modeling.emf.diffmerge, 2024.

[21] "EMF Compare," https://eclipse.dev/emf/compare/, Eclipse Foundation, 2024.

[22] W. Bian, O. Alam, and J. Kienzle, "Automated Grading of Class Diagrams," in *2019 ACM/IEEE 22nd International Conference on Model Driven Engineering Languages and Systems Companion (MODELS-C)*. IEEE, 2019, pp. 700–709.

[23] M. Hosseinibaghdadabadi, O. Alam, N. Almerge, and J. Kienzle, "Automated Grading of Use Cases," in *2023 ACM/IEEE 26th International Conference on Model Driven Engineering Languages and Systems (MODELS)*. IEEE, 2023, pp. 106–116.

[24] P. Singh, Y. Boubekeur, and G. Mussbacher, "Detecting Mistakes in a Domain Model," in *Proceedings of the 25th International Conference on Model Driven Engineering Languages and Systems Companion (MODELS-C)*, 2022, pp. 257–266.

[25] K. Chen, B. Chen, Y. Yang, G. Mussbacher, and D. Varró, "Embedding-based Automated Assessment of Domain Models," in *Proceedings of the ACM/IEEE 27th International Conference on Model Driven Engineering Languages and Systems Companion (MODELS-C)*, 2024, pp. 87–94.

[26] E. Triandini, R. Fauzan, D. O. Siahaan, and S. Rochimah, "Sequence Diagram Similarity Measurement: A Different Approach," in *2019 16th International Joint Conference on Computer Science and Software Engineering (JCSSE)*. IEEE, 2019, pp. 348–351.

[27] C.-H. Chiang and H.-y. Lee, "Can Large Language Models Be an Alternative to Human Evaluations?" in *Proc. of the Annual Meeting of the Association for Computational Linguistics (ACL)*, 2023, pp. 15 607–15 631.

[28] J. Wang, Y. Liang, F. Meng, Z. Sun, H. Shi, Z. Li, J. Xu, J. Qu, and J. Zhou, "Is ChatGPT a Good NLG Evaluator? A Preliminary Study," in *Proc. of the New Frontiers in Summarization Workshop (NewSum)*, 2023, pp. 1–11.

[29] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. P. Xing, H. Zhang, J. E. Gonzalez, and I. Stoica, "Judging LLM-as-a-judge with MT-bench and Chatbot Arena," in *Proc. of the International Conference on Neural Information Processing Systems (NeurIPS)*, 2024.

[30] J. Fu, S.-K. Ng, Z. Jiang, and P. Liu, "GPTScore: Evaluate as You Desire," in *Proc. of the Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*, 2024, pp. 6556–6576.

[31] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, "Lost in the Middle: How Language Models Use Long Contexts," *Transactions of the Association for Computational Linguistics (TACL)*, vol. 12, pp. 157–173, 2024.

[32] K. Ahmed, J. Song, B. Chen, O. Wei, and B. Zhang, "MCeT," https://github.com/Huawei-TTE/MCeT, 2025.

[33] "PlantUML," https://plantuml.com/, 2025.

[34] A. Plaat, A. Wong, S. Verberne, J. Broekens, N. van Stein, and T. Back, "Reasoning with large language models, a survey," *arXiv preprint arXiv:2407.11511*, 2024.

[35] J. Gu, X. Jiang, Z. Shi, H. Tan, X. Zhai, C. Xu, W. Li, Y. Shen, S. Ma, H. Liu *et al.*, "A Survey on LLM-as-a-Judge," *arXiv preprint arXiv:2411.15594*, 2024.

[36] H. Li, Q. Dong, J. Chen, H. Su, Y. Zhou, Q. Ai, Z. Ye, and Y. Liu, "LLMs-as-Judges: A Comprehensive Survey on LLM-based Evaluation Methods," *arXiv preprint arXiv:2412.05579*, 2024.

[37] Y.-T. Lin and Y.-N. Chen, "LLM-Eval: Unified Multi-Dimensional Automatic Evaluation for Open-Domain Conversations with Large Language Models," in *Proceedings of the 5th Workshop on NLP for Conversational AI (NLP4ConvAI 2023)*, 2023, pp. 47–58.

[38] Y. Liu, D. Iter, Y. Xu, S. Wang, R. Xu, and C. Zhu, "G-Eval: NLG Evaluation using Gpt-4 with Better Human Alignment," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023, pp. 2511–2522.

[39] M. Renze and E. Guven, "Self-Reflection in LLM Agents: Effects on Problem-Solving Performance," *arXiv preprint arXiv:2405.06682*, 2024.

[40] L. Pan, M. Saxon, W. Xu, D. Nathani, X. Wang, and W. Y. Wang, "Automatically Correcting Large Language Models: Surveying the Landscape of Diverse Automated Correction Strategies," *arXiv preprint arXiv:2308.03188*, 2023.

[41] T. Yue, L. C. Briand, and Y. Labiche, "Facilitating the Transition from Use Case Models to Analysis Models: Approach and Experiments," *Transactions on Software Engineering and Methodology (TOSEM)*, vol. 22, no. 1, pp. 1–38, 2013.

[42] Q. Dong, L. Li, D. Dai, C. Zheng, J. Ma, R. Li, H. Xia, J. Xu, Z. Wu, B. Chang, X. Sun, L. Li, and Z. Sui, "A Survey on In-

Authorized licensed use limited to: Institut Teknologi Sepuluh Nopember. Downloaded on September 29,2026 at 00:44:30 UTC from IEEE Xplore. Restrictions apply.

context Learning," in *Proc. of Empirical Methods in Natural Language Processing (EMNLP)*, 2024, pp. 1107–1128.
[43] J. Wei, X. Wang, D. Schuurmans, M. Bosma, b. ichter, F. Xia, E. Chi, Q. V. Le, and D. Zhou, "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models," in *Proc. of the International Conference on Neural Information Processing Systems (NeurIPS)*, vol. 35, 2022, pp. 24 824–24 837.
[44] L. Chen, J. Davis, B. Hanin, P. Bailis, I. Stoica, M. Zaharia, and J. Zou, "Are More LLM Calls All You Need? Towards the Scaling Properties of Compound AI Systems," in *Proc. of the International Conference on Neural Information Processing Systems (NeurIPS)*, vol. 37, 2024, pp. 45 767–45 790.
[45] X. Chen, R. Aksitov, U. Alon, J. Ren, K. Xiao, P. Yin, S. Prakash, C. Sutton, X. Wang, and D. Zhou, "Universal Self-Consistency for Large Language Models," in *ICML 2024 Workshop on In-Context Learning*, 2024.
[46] T. Cai, Z. Tan, X. Song, T. Sun, J. Jiang, Y. Xu, Y. Zhang, and J. Gu, "FoRAG: Factuality-optimized Retrieval Augmented Generation for Web-enhanced Long-form Question Answering," in *Proc. of the Conference on Knowledge Discovery and Data Mining (KDD)*, 2024, pp. 199–210.
[47] "Cyber-physical V&V challenges for the evaluation of State of the art model checkers," https://github.com/hbourbouh/lm_challenges.
[48] A. Ferrari, G. O. Spagnolo, and S. Gnesi, "PURE: A Dataset of Public Requirements Documents," in *Proc. of International Requirements Engineering Conference (RE)*, 2017, pp. 502–505.
[49] F. Dalpiaz and A. Sturm, "Conceptualizing Requirements Using User Stories and Use Cases: A Controlled Experiment," in *Proc. of Requirements Engineering: Foundation for Software Quality (REFSQ)*, 2020, pp. 221–238.
[50] "GPT-4o mini: advancing cost-efficient intelligence," https://openai.com/index/gpt-4o-mini-advancing-cost-efficient-intelligence/, OpenAI, 2024.
[51] "GPT-4o System Card," https://openai.com/index/gpt-4o-system-card/, OpenAI, 2024.
[52] DeepSeek-AI, A. Liu, B. Feng, B. Xue, B. Wang, B. Wu, C. Lu, C. Zhao, C. Deng, C. Zhang, C. Ruan *et al.*, "DeepSeek-V3 Technical Report," 2025. [Online]. Available: https://arxiv.org/abs/2412.19437
[53] DeepSeek-AI, D. Guo, D. Yang, H. Zhang, J. Song, R. Zhang, R. Xu, Q. Zhu, S. Ma, P. Wang, X. Bi *et al.*, "DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning," 2025. [Online]. Available: https://arxiv.org/abs/2501.12948
[54] J. Cohen, "A Coefficient of Agreement for Nominal Scales," *Educational and Psychological Measurement (EPM)*, vol. 20, no. 1, pp. 37–46, 1960.
[55] T. Zhang, V. Kishore, F. Wu, K. Q. Weinberger, and Y. Artzi, "BERTScore: Evaluating Text Generation with BERT," in *International Conference on Learning Representations*, 2020.
[56] Q. Jia, S. Ren, Y. Liu, and K. Zhu, "Zero-shot Faithfulness Evaluation for Text Summarization with Foundation Language Model," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023, pp. 11 017–11 031.
[57] C.-M. Chan, W. Chen, Y. Su, J. Yu, W. Xue, S. Zhang, J. Fu, and Z. Liu, "ChatEval: Towards Better LLM-based Evaluators through Multi-Agent Debate," in *The International Conference on Learning Representations*, 2024.
[58] S. Liang, B. Zhang, J. Zhao, and K. Liu, "ABSEval: An Agent-based Framework for Script Evaluation," in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, 2024, pp. 12 418–12 434.
[59] R. Wang, J. Guo, C. Gao, G. Fan, C. Y. Chong, and X. Xia, "Can LLMs Replace Human Evaluators? An Empirical Study of LLM-as-a-Judge in Software Engineering," *arXiv preprint arXiv:2502.06193*, 2025.
[60] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language Agents with Verbal Reinforcement Learning," *Advances in Neural Information Processing Systems*, vol. 36, pp. 8634–8652, 2023.
[61] A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegreffe, U. Alon, N. Dziri, S. Prabhumoye, Y. Yang *et al.*, "Self-Refine: Iterative Refinement with Self-Feedback," *Advances in Neural Information Processing Systems*, vol. 36, pp. 46 534–46 594, 2023.
[62] R. W. Hasker and M. Rowe, "UMLint: Identifying Defects in UML Diagrams," in *2011 ASEE Annual Conference and Exposition*, 2011.
[63] Y. Boubekeur, G. Mussbacher, and S. McIntosh, "Automatic Assessment of Students' Software Models Using a Simple Heuristic and Machine Learning," in *Proceedings of the 23rd ACM/IEEE International Conference on Model Driven Engineering Languages and Systems Companion (MODELS-C)*, 2020, pp. 1–10.
[64] D. R. Stikkolorum, P. van der Putten, C. Sperandio, and M. Chaudron, "Towards Automated Grading of UML Class Diagrams with Machine Learning," *Proceedings of the 31st Benelux Conference on Artificial Intelligenc (BNAIC 2019) and the 28th Belgian Dutch Conference on Machine Learning (Benelearn 2019)*, vol. 2491, 2019.
[65] J. Di Rocco, D. Di Ruscio, C. Di Sipio, P. T. Nguyen, and R. Rubei, "On the use of Large Language Models in Model-Driven Engineering," *Software and Systems Modeling*, pp. 1–26, 2025.
[66] K. Chen, Y. Yang, B. Chen, J. A. H. López, G. Mussbacher, and D. Varró, "Automated Domain Modeling with Large Language Models: A Comparative Study," in *2023 ACM/IEEE 26th International Conference on Model Driven Engineering Languages and Systems (MODELS)*. IEEE, 2023, pp. 162–172.
[67] S. Arulmohan, M.-J. Meurs, and S. Mosser, "Extracting Domain Models from Textual Requirements in the Era of Large Language Models," in *2023 ACM/IEEE International Conference on Model Driven Engineering Languages and Systems Companion (MODELS-C)*. IEEE, 2023, pp. 580–587.
[68] M. B. Chaaben, L. Burgueño, and H. Sahraoui, "Towards using Few-Shot Prompt Learning for Automating Model Completion," in *2023 IEEE/ACM 45th international conference on software engineering: New ideas and emerging results (ICSE-NIER)*. IEEE, 2023, pp. 7–12.
[69] L. Apvrille and B. Sultan, "System Architects Are not Alone Anymore: Automatic System Modeling with AI," in *MODELSWARD 2024: 12th International Conference on Model-Based Software and Systems Engineering*, vol. 1. SCITEPRESS-Science and Technology Publications, 2024, pp. 27–38.
[70] T. Buchmann, "Prompting Bidirectional Model Transformations-The Good, The Bad and The Ugly," in *Companion Proceedings of the ACM/IEEE 27th International Conference on Model Driven Engineering Languages and Systems*, 2024, pp. 550–555.
[71] J. A. H. López, M. Földiák, and D. Varró, "Text2VQL: Teaching a Model Query Language to Open-Source Language Models with ChatGPT," in *Proceedings of the ACM/IEEE 27th International Conference on Model Driven Engineering Languages and Systems*, 2024, pp. 13–24.
[72] S. Abukhalaf, M. Hamdaqa, and F. Khomh, "On Codex Prompt Engineering for OCL Generation: An Empirical Study," in *2023 IEEE/ACM 20th International Conference on Mining Software Repositories (MSR)*. IEEE, 2023, pp. 148–157.

Authorized licensed use limited to: Institut Teknologi Sepuluh Nopember. Downloaded on September 29,2026 at 00:44:30 UTC from IEEE Xplore. Restrictions apply.